PAT: Accelerating LLM Decoding via Prefix-Aware Attention with Resource Efficient Multi-Tile Kernel

Jinjun Yi[†]
Tianjin University
Tianjin, China
march_h@tju.edu.cn

Zhixin Zhao[†]
Tianjin University
Tianjin, China
zhao612@tju.edu.cn

Yitao Hu[*]
Tianjin University
Tianjin, China
yitao@tju.edu.cn

Ke Yan
Tianjin University
Tianjin, China
yank@tju.edu.cn

Weiwei Sun
Tianjin University
Tianjin, China
sww@tju.edu.cn

Hao Wang
Stevens Institute of Technology
Hoboken, NJ, USA
hwang9@stevens.edu

Laiping Zhao
Tianjin University
Tianjin, China
laiping@tju.edu.cn

Yuhao Zhang
Tianjin University
Tianjin, China
yuhaozhang@tju.edu.cn

Wenxin Li
Tianjin University
Tianjin, China
toliwenxin@tju.edu.cn

Keqiu Li
Tianjin University
Tianjin, China
keqiu@tju.edu.cn

Abstract

LLM serving is increasingly dominated by decode attention, which is a memory-bound operation due to massive KV cache loading from global memory. Meanwhile, real-world workloads exhibit substantial, hierarchical shared prefixes across requests (*e.g.,* system prompts, tools/templates, RAG). Existing attention implementations fail to fully exploit prefix sharing: *one-query-per-CTA* execution repeatedly loads shared prefix KV cache, while *one-size-fits-all* tiling leaves on-chip resources idle and exacerbates bubbles for uneven KV lengths. These choices amplify memory bandwidth pressure and stall memory-bound decode attention.

This paper introduces PAT, a prefix-aware attention kernel implementation for LLM decoding that organizes execution with a pack-forward-merge paradigm. PAT packs queries by shared prefix to reduce repeated memory accesses, runs a customized multi-tile kernel to achieve high resource efficiency. It further applies practical multi-stream forwarding and KV splitting to reduce resource bubbles. The final merge performs online softmax with negligible overhead. We implement PAT as an off-the-shelf plugin for vLLM. Evaluation on both real-world and synthetic workloads shows that PAT reduces attention latency by 53.5% on average and TPOT by 17.0-93.1% under the same configurations against state-of-the-art attention kernels. PAT's source code is publicly available at https://github.com/flashserve/PAT.

CCS Concepts: • **Computing methodologies** → **Machine learning**; • **Computer systems organization** → **Cloud computing**.

Keywords: Large Language Models; LLM Inference; Prefix Aware Attention; GPU Kernel Scheduling

ACM Reference Format:
Jinjun Yi, Zhixin Zhao, Yitao Hu, Ke Yan, Weiwei Sun, Hao Wang, Laiping Zhao, Yuhao Zhang, Wenxin Li, and Keqiu Li. 2026. PAT: Accelerating LLM Decoding via Prefix-Aware Attention with Resource Efficient Multi-Tile Kernel. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26), March 21–26, 2026, Pittsburgh, PA, USA.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3779212.3790200

1 Introduction

Transformer-based Large Language Models (LLMs) have rapidly advanced across diverse domains, including conversational assistants [7, 10], code generation [4, 12], retrieval-augmented generation (RAG) [18, 48], and agent/tool workflows [22, 53]. As deployment grows, latency and resource efficiency of online inference have become increasingly critical. While prior work has explored optimizations in memory management [16, 45], scheduling [39, 57], quantization [6, 20], and architecture [21, 58], there remain new challenges and opportunities. We highlight two emerging trends below.

Longer contexts and outputs. Context length has scaled to millions of tokens [2], while techniques like Chain-of-Thought (CoT) [43] demand longer outputs. As a result, decode attention (*i.e.,* attention operations in the decoding

[†]Both authors contributed equally to this work.
[*]Corresponding author.

ASPLOS '26, Pittsburgh, PA, USA.

ACM ISBN 979-8-4007-2359-9/2026/03
https://doi.org/10.1145/3779212.3790200

stage) increasingly dominates end-to-end latency due to repeated loading of growing KV caches from global to on-chip memory, which poses a memory-bound challenge.

Shared prefixes across requests. System prompts, RAG documents, and agent templates introduce multi-level shared prefixes across requests, creating an optimization opportunity. Existing systems [16, 55] implement prefix KV cache reuse, which reduces memory footprint by storing and reusing KV cache across requests. However, it can *not* reduce global memory accesses, which is the bottleneck for decode attention, thereby leading to higher attention latency.

Recent attention kernel implementations aim to reduce attention latency [33, 36, 38, 49, 50, 60], but still face redundant KV cache loads from global memory and inefficient resource utilization. *Query-centric* kernels [8, 9, 38, 50] follow a one-query-per-CTA strategy to map each query and corresponding KV cache into an independent Cooperative Thread Array (CTA; also known as a thread block) for execution on the GPU, causing redundant global memory loads for shared prefixes. *KV-centric* kernels [33, 47, 49, 60] instead pack the KV cache of cross-query shared prefix into one CTA to reduce redundant memory accesses, but adopt a one-size-fits-all design that fixes the tile size of the GPU kernel [25] and applies padding to fill the tile, which wastes on-chip memory and limits CTA concurrency.

Based on the analysis, we argue that **an efficient attention kernel implementation for LLM decoding should be memory-oriented with prefix-aware execution**, so as to reduce redundant memory accesses and maintain high resource efficiency. However, the dynamicity of workloads makes this design non-trivial. First, the combination of multi-level shared prefixes and the dynamic join-and-leave nature of continuous batching makes the structure of shared KV caches in a decode batch (*i.e.,* batched queries in decode stage) variable across decode steps. The attention kernel must pack the decode batch into CTAs effectively, so as to reduce redundant global memory accesses with low runtime overhead. Second, the autoregressive nature of LLMs makes queries have diverse KV lengths, and the number of queries that share the same KV blocks in one CTA varies over time. These features bring dynamic hardware resource requirements per CTA, which in turn affect the memory bandwidth usage and GPU utilization. The attention kernel must adapt to the dynamicity to achieve high resource efficiency.

To address these challenges, we present PAT, an attention kernel implementation for LLM decode stage with a *pack–forward–merge* execution paradigm. Specifically, in the *pack stage* (§5), PAT employs a prefix-aware pack scheduler and a lazy update mechanism, which efficiently pack decode batches into CTAs to reduce redundant global memory accesses with negligible overhead. It further designs a multi-tile kernel based on resource-efficiency analysis, so as to adapt to CTA dynamicity and avoid on-chip memory waste. In the *forward stage* (§6), PAT designs a multi-stream forward and long-KV split strategy, which parallelizes the multi-tile kernel across multiple CUDA streams and splits CTAs with excessively long KV lengths. These designs enable PAT to eliminate execution bubbles and achieve high global memory bandwidth utilization. In the *merge stage* (§7), PAT applies a lightweight kernel with online softmax [9], which merges partial results across CTAs for each query.

In summary, this paper makes the following contributions:

1. We provide an in-depth analysis of the bottlenecks in decode attention (§2) and the optimization opportunities and challenges introduced by shared prefixes (§3), from both hardware characteristics and the attention execution pipeline perspectives.
2. We design and implement a prefix-aware attention kernel PAT for LLM decoding. It incorporates several novel designs within the pack–forward–merge paradigm to reduce redundant global memory accesses and achieve efficient hardware utilization (§4-§7).
3. We evaluate PAT on synthetic and real-world workloads, demonstrating that compared with state-of-the-art baselines, it reduces attention latency by 53.5% on average under the same decode batches (§8.3) and lowers TPOT by 17.0-93.1% under the same request rate (§8.4).

2 Background

2.1 LLM Inference and Attention

Transformer-based Large Language Models (LLMs) follow an autoregressive inference process with two phases: prefill and decode. In the prefill phase, the model performs a full forward pass over the input sequence to generate the first token. The decode phase then iteratively generates one token at a time until reaching an end-of-sequence token or a preset length. Each decode step involves three major computations: Self-Attention, Query-Key-Value-Output (QKVO) projection, and Multi-Layer Perceptron (MLP). Among these, Self-Attention is central for modeling global dependencies as:

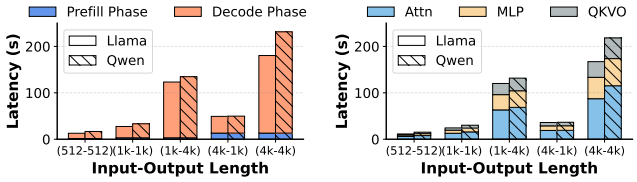
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

During decoding, the query Q from the current token attends to all previous keys K and values V. Since K and V do not change across steps, they are stored in global memory using a Key-Value (KV) Cache, reducing redundant computation but increasing memory footprint.

2.2 Memory Bottleneck of Attention

Driven by the inference scaling law [44], LLMs are trending toward *longer contexts* and *longer outputs*. For instance, Llama-4 supports up to 10 million input tokens [2], and Chain-of-Thought prompting [43] has significantly increased output lengths for complex reasoning.

Although KV Caching avoids repeated computation, it has shifted the bottleneck from compute to memory access:



(a) end-to-end latency **(b)** decode phase latency

Figure 1. Latency breakdown of Llama-3-8B and Qwen3-8B across context length on A100, vLLM v0.9.0, batch size 64

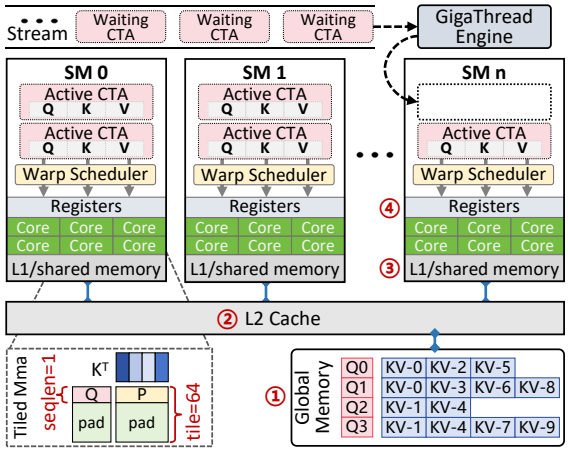


Figure 2. GPU architecture of typical NVIDIA GPUs.

each decode step must fetch a growing amount of KV cache from global memory to on-chip memory [5, 54]. As input and output lengths grow, this access cost dominates inference latency. As in Figure 1, decode attention can contribute up to 53% of the total latency for Llama-3-8B and Qwen3-8B.

2.3 GPU Execution Model

To fully understand the decode attention bottleneck, we should examine the GPU's hardware and software architecture, as well as the execution process of the attention kernel. As shown in Figure 2, the GPU hardware architecture is composed of ① **global memory**, ② **L2 Cache**, an array of Streaming Multiprocessors (SMs), and a global hardware scheduler (GigaThread Engine). Each SM is a basic computational unit containing various CUDA Cores, Tensor Cores, and its own hierarchical memory structure: a programmer-managed ③ **Shared Memory / L1 Cache** that enables low-latency data exchange among threads within a Cooperative Thread Array (CTA, aka Thread Block), and a ④ **Register File** for thread-private storage. As shown in Table 1, the GPU memory hierarchy involves a significant trade-off between size and speed, with global memory access being orders of magnitude slower than on-chip memory access.

When executing an attention kernel, the workload is divided into CTAs, scheduled across SMs. During this process, the required KV cache data is transferred from *slow global memory* up the hierarchy into *fast on-chip shared memory*, where it is finally loaded into registers for computation. State-of-the-art implementations [8, 9, 50] are designed to exploit this memory hierarchy. For example, FlashAttention [9, 25] partitions K/V caches into small tiles along the sequence length (Figure 3a) and processes them in a pipelined fashion: while one tile is computed, the next is prefetched asynchronously (Figure 3b), reducing memory latency.

Level	Shared By	Size	Latency	Bandwidth*	Type
Register	Thread	256KB/SM†	~2ns	~20 TB/s	on-chip
Shared Memory / L1 Cache	CTA	192KB/SM‡	~20ns	~19 TB/s	on-chip
L2 Cache	All SMs	40MB	~140ns	~2 TB/s	on-chip
Global Memory	All SMs	80GB	~200ns	~2 TB/s	off-chip

*Read/write bandwidth from the upper memory level.
†Each thread is limited to 255 registers (4*8 bits each).
‡Each CTA can address up to 163 KB of shared memory.

Table 1. Memory hierarchy of A100-SXM4-80GB [1, 23].

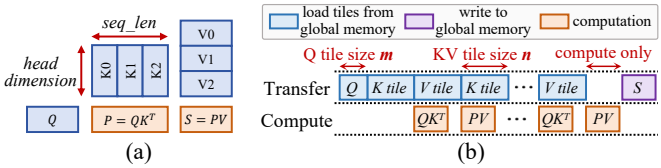


Figure 3. (a) Two General Matrix-Vector multiplications (GEMV) in attention. (b) Tiling execution pipeline.

Nonetheless, decode attention remains bottlenecked by two challenges: limited bandwidth between global and on-chip memory, and low arithmetic intensity due to heavy KV cache loading. Thus, further optimization must follow two principles: (1) *Reduce KV cache transfer loads from global memory*, and (2) *Fully utilize available memory bandwidth.*

3 Motivation

In this section, we first characterize the shared-prefix pattern commonly observed in LLM workloads (§3.1) and identify two limitations in existing attention implementations under this pattern: redundant global memory accesses (§3.2) and low hardware utilization (§3.3). We then introduce the pack–forward–merge paradigm to address these problems (§3.4) and analyze the corresponding challenges (§3.5).

3.1 Shared Prefixes and Prefix Reuse

Shared prefixes between requests are common and hierarchical in modern LLM workloads [13, 32, 42]. As shown in Figure 4, our analysis of four real-world traces [34, 41] finds a prefix ratio of 51.9 – 75.0%, meaning that more than half of KV cache tokens come from prefixes reused across requests. Under continuous batching, this cross-request sharing leads to *multi-level intra-batch prefixes*. On the Conversation and ToolAgent traces, using the setup in §8.2, intra-batch shared

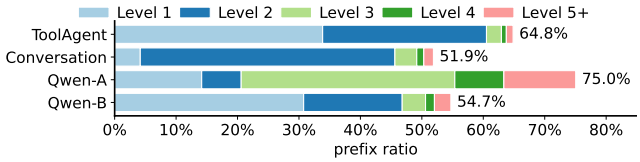


Figure 4. Prefix ratio of four traces: ToolAgent: tool and agent interaction workload [34]; Conversation: online conversation workload [34]; Qwen-A: online API service [41]; Qwen-B: task automation with API calling workload [41].

prefixes cover 2.8 – 82.6% of KV caches, and each batch ends up with 2.72 distinct shared prefixes on average. These significant and hierarchical shared prefixes present unique opportunities and challenges for LLM inference optimization.

Systems like SGLang[55] and vLLM[16] adopt *prefix reuse* by mapping shared logical prefixes to a single physical copy in global memory. This approach reuses KV caches across requests to reduce memory usage. However, it can *not* leverage intra-batch shared prefixes to reduce global memory accesses, which remain the bottleneck. To understand why, we analyze two key inefficiencies in existing approaches under shared-prefix scenarios: redundant memory access (§3.2) and underutilized hardware resources (§3.3).

3.2 Redundant Memory Access

A major inefficiency in current attention kernels stems from redundant memory accesses. To execute a decode batch, attention kernels use a packing strategy that groups queries and their KV caches into CTAs. In the decode batch with 4 queries (Figure 5a), existing kernels' query-centric paradigm adopts a *one-query-per-CTA* packing strategy [36, 50], where each query and its KV are independently assigned to a CTA (Figure 5b). While simple to schedule, this strategy causes shared KV prefixes (*e.g.,* KV-0, KV-1) to be repeatedly loaded from slow global memory into on-chip memory. Although L2 cache offers partial reuse, it is limited by size and bandwidth.

To examine the inefficiency of the one-query-per-CTA strategy, we profile FlashAttention [36] using ncu [31], comparing its KV cache traffic with the theoretical minimum (where each shared block is loaded once), and with PAT. As shown in Figure 6a, FlashAttention incurs 4.3-8.7× more KV cache than the theoretical minimum, and 4.1–7.5× more than PAT. This result confirms significant overhead from redundant memory access for the one-query-per-CTA paradigm.

Observation #1: Existing query-centric attention kernels suffer from substantial memory access redundancy due to their one-query-per-CTA execution paradigm.

3.3 Two-Dimension Resource Inefficiencies

Furthermore, existing attention kernels exhibit inefficient hardware utilization. Both query-centric and KV-centric designs use a tiled pipeline (Figure 3) to overlap memory access

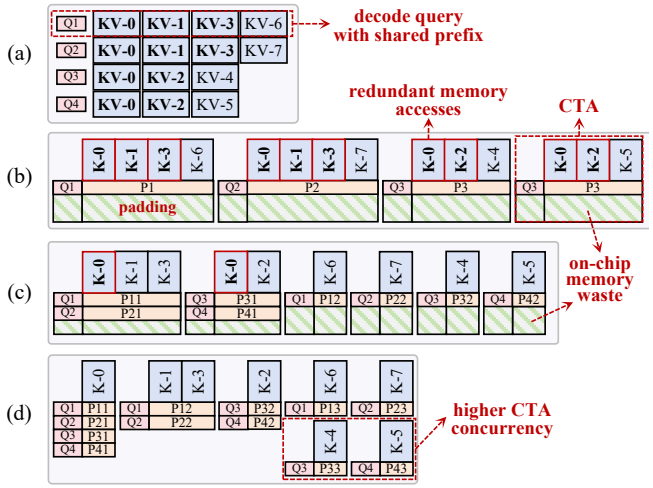
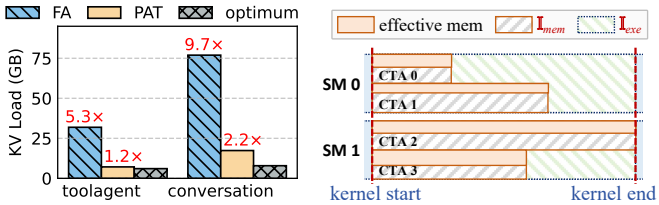


Figure 5. Comparison of packing strategies. (a) A decode batch of 4 queries with shared prefixes. (b) Query-centric packing causes redundant memory access. (c) KV-centric packing causes memory waste. (d) Memory-centric prefix-aware packing avoids redundancy and improves utilization.



(a) redundant memory accesses **(b)** resource inefficiencies

Figure 6. Limitations of existing attention kernels. (a) Average KV cache load from global memory per decode step on the *toolagent* and *conversation* traces, comparing FlashAttention (FA), PAT, and the theoretical optimum. (b) Two-dimensional resource inefficiencies.

and computation. In this paradigm, selecting appropriate tile sizes (*i.e.,* query tile size m and KV tile size n from Figure 3b) is critical for resource efficiency [9]. However, existing kernels adopt a *one-size-fits-all* design, which specifies a single, hard-coded tile size for all CTAs (*e.g.,* $m = 64, n = 32$ [36]).

As shown in Figure 5b-c, this static approach ignores the dynamic nature of LLM workloads, leading to significant resource inefficiencies along two dimensions[1] in Figure 6b: (1) Memory Waste ($\mathbb{I}_{mem}$): When fewer than m queries share a KV prefix, CTAs must pad inputs, wasting shared memory and registers to store unused data. (2) Execution Bubble ($\mathbb{I}_{exe}$): Due to varying KV lengths across CTAs, fixed-size tiling causes imbalanced workloads, leaving SMs underutilized in the tail stages of execution.

[1]While specific prior works [33] design one more query tile sizes m, it is still insufficient for handling highly dynamic query numbers and KV lengths.

Observation #2: Existing query-centric and kv-centric attention kernels suffer from two-dimension resource inefficiencies due to their one-size-fits-all execution paradigm.

3.4 Insight: Pack-forward-merge Paradigm

Building on the observations above, we identify two design principles for efficient decode attention (Figure 5d): (1) Intra-CTA KV Cache Sharing: pack queries with shared KV prefixes into the same CTA to enable KV reuse in shared memory and avoid redundant global memory access. (2) Resource-Efficient Kernel Design: tailor kernel implementations to GPU architecture and CTA configurations to sustain high memory bandwidth and minimize resource inefficiencies.

Following these principles, we propose *pack–forward–merge paradigm* as follows: (1) **Pack**: group queries by shared prefix into CTAs to eliminate redundant KV loads; (2) **Forward**: execute CTAs using resource-optimized kernels that output partial results in tiles; (3) **Merge**: apply online softmax to combine partial results into final outputs.

3.5 Challenges

Although the pack–forward–merge paradigm directly targets the objectives of reducing global memory access and improving resource utilization, it faces two major challenges:

Challenge 1: Packing complexity. LLM inference often involves deep, multi-level shared prefixes and long contexts, significantly expanding the packing search space. Each prefix level may yield multiple packing candidates with different trade-offs. Additionally, continuous batching introduces dynamic request changes, requiring frequent packing updates. An effective strategy must account for both prefix hierarchy and batch dynamism, generating CTA assignments with minimal latency (§5.1).

Challenge 2: Workload variability. Autoregressive decoding leads to large variation in KV lengths. Grouping queries by shared prefixes further amplifies variation in CTA sizes, ranging from one to dozens of queries. This variability affects resource demands and execution time across CTAs. The forward stage must use a kernel design that adapts to both hardware and workload characteristics (§5.2) and employs scheduling strategies to minimize time bubbles (§6).

4 Overview

To address these challenges, we design PAT, a *memory-centric attention kernel implementation* that follows the pack-forward-merge paradigm. It serves as a backend for the serving system vLLM [16]. In the *pack* stage, PAT adopts a profit-model-based heuristic packing strategy to aggregate queries that share KV into a CTA, so as to mitigate redundant global memory accesses (§5.1). It further designs multi-tile kernels and a runtime tile selector to choose an efficient kernel for each CTA (§5.2). In the *forward* stage, PAT adopts multi-stream execution and a long-KV split strategy to enable efficient kernel execution, which reduces execution bubbles (§6). Finally, in the *merge* stage, PAT uses a lightweight kernel based on online softmax to merge each query's intermediate results across CTAs and produce the final output (§7).

5 Pack Scheduler

We first introduce a pack scheduler that packs a decode batch into CTAs by shared prefixes to reduce redundant global memory accesses (§5.1), and then present the customized multi-tile kernels that efficiently execute these CTAs (§5.2).

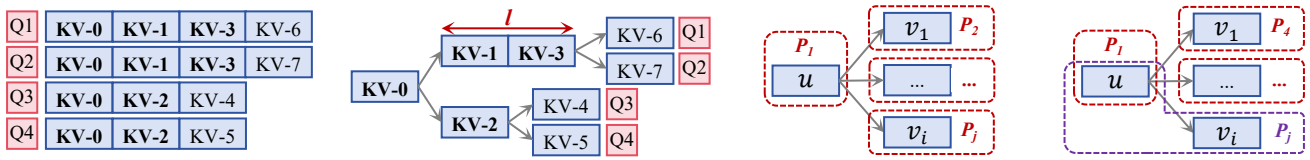
5.1 Pack Scheduler

Insight and Approach. As noted in §3.2, the one-query-per-CTA paradigm repeatedly loads KV blocks for shared prefixes, worsening the memory bottleneck of decode attention. We therefore introduce a heuristic pack scheduler that (i) abstracts a decode batch's block table into a prefix tree, (ii) scores candidate packing schemes with a memory-centric profit model, and (iii) packs the decode batch into memory-optimized CTAs to cut redundant global accesses.

Problem Formulation Since decode attention is memory-bound (§2.3 and §3.2), the pack stage aims to minimize global memory accesses for a given decode batch (Figure 7a). The *input* is a batch of queries plus a block table where each row lists block IDs for a query; a shared prefix appears as identical leading block IDs across rows. The *output* is a partition $\mathcal{P} = \{P_1, P_2, \dots\}$ of CTAs, where each P_i packs queries sharing successive identical prefix blocks (queries may be split across CTAs). We seek $\mathcal{P}^*$ that minimizes memory accesses, comprised of loaded KV cache size and per-query intermediate reads/writes due to splits/merges. The search space grows exponentially with query count and prefix lengths [33], so an exact solver is impractical for online serving, and we turn to a heuristic pack scheduler described next.

Tree Structure Block Table. For efficient implementation, the pack scheduler first converts the two-dimensional block table (Figure 7a) into a tree structure block table (Figure 7b). Each internal node represents a shared prefix of KV blocks. It has two attributes: (1) l, the KV-cache length of this shared prefix, and (2) s, the number of queries that share it. Each leaf corresponds to one query, and the path from the root to that leaf reconstructs the query's full KV cache blocks.

Intra-node profit. We first discuss the profit and overhead of packing the queries within a single non-leaf node u (with attributes l_u and $s_u > 1$) into one CTA. First, compared with the one-query-per-CTA paradigm, packing s_u queries with shared KV length of l_u into a single CTA could reduce the KV cache loads from s_u times to 1 time. Therefore, it saves $(s-1)\, l_u\, d$ global memory accesses (d is the head dimension). However, packing the shared prefixes of all queries into a CTA produces $2\, s_u$ times (half comes from node u and the



(a) a decode batch (b) tree structure block table (c) scheme 1: split (d) scheme 2: merge

Figure 7. Workflow of the pack scheduler: (a) An input decode batch with 4 queries. (b) Tree structure block table. (c) Packing scheme 1 that splits leaf nodes with the parent node. (d) Packing scheme 2 that merges specific leaf node with the parent node.

other half from leaf nodes) per-query intermediate writes and reads. Therefore the overhead of memory access[2] is $2 \times (2\, s_u\, d + 2\, s_u\, d) = 8\, s_u\, d$. Then, the profit–overhead ratio for packing a non-leaf node to a CTA is:

$$r = \frac{(s_u - 1)\, l_u\, d}{8\, s_u\, d} \geq \frac{l_u}{16}.$$

In practice, the length of shared KV $l_u \geq 16$ since sharing is performed at the granularity of KV blocks [16, 36], whose sizes are typically larger than 16. Therefore, *packing a node into a CTA yields a positive profit.*

Inter-node profit. When child nodes are involved, the profit and overhead change. Let $\{v_1, v_2, ...v_i\}$ be the children of u, where child v_i has KV length l_i and queries s_i ($s_u = \sum_i s_i$). We compare two schemes in Figure 7c-d.

Scheme 1: Split. As in Figure 7c, a naive packing scheme is splitting each node into an individual CTA. Following the analysis above, we can derive the overall profit as:

$$\underbrace{(s_u - 1)\, l_u\, d}_{\text{profit of } u} - \underbrace{4 s_u d}_{\text{overhead of } u} + \underbrace{\textstyle\sum_i (s_i - 1)\, l_i\, d}_{\text{profit of } \{v_1, v_2, ...v_i\}}. \quad (1)$$

Scheme 2: Merge. As shown in Figure 7d, when considering child nodes, we can pack specific child node v_i and parent node u into a CTA to eliminate their intermediate results. In this case, the number of queries associated with node u becomes $s'_u = s_u - s_i$, while the KV length l_u remains unchanged. Then the profit of the unmerged part could also be estimated using Equation 1, and the profit of the merged part $u \sim v_i$ becomes $(s_i - 1)\,(l_u + l_i)\, d$. Therefore, the overall profit is:

$$\underbrace{(s_u - s_i - 1)\, l_u\, d}_{\text{profit of } u} - \underbrace{4(s_u - s_i) d}_{\text{overhead of } u} + \underbrace{\textstyle\sum_{k \neq i} (s_k - 1)\, l_k\, d}_{\text{profit of } \{v_1, v_2, ...v_{i-1}\}} + \underbrace{(s_i - 1)\,(l_u + l_i)\, d}_{\text{profit of } u \sim v_i}. \quad (2)$$

Scheme comparison. The incremental profit of Scheme 2 over Scheme 1 is $4 s_i d - l_u d$. Hence Scheme 2 is preferred when $4 s_j > l_u$. When the shared prefix at u is short and the specific child node v_i's queries are large enough, merging them achieves higher profit by eliminating unnecessary intermediate results. Otherwise, keep u and v_i separate.

Pack Scheduler. We implement a heuristic scheduler guided by the profit analysis to pack the decode batch into CTAs. It first converts the block table into a forest, where each root is a unique first-level prefix and each root–leaf path encodes a query's multi-level shared prefixes plus its non-shared suffix. Each node stores the shared block IDs and the query IDs. For each tree, the scheduler invokes *TreeHeuristic* (Algorithm 1) to produce CTAs and adds them to $\mathcal{P}$. *TreeHeuristic* packs each leaf as an independent CTA (line 4), scans the children of each internal node, applies the inter-node profit model to choose a scheme, and recursively packs children (lines 5–13). It then packs the node's remaining queries into a CTA and returns with its children's CTAs (lines 14–15). This yields memory-efficient CTAs with fewer global-memory accesses and linear complexity $O(|V|+|E|)$ since each node and edge is processed once[3].

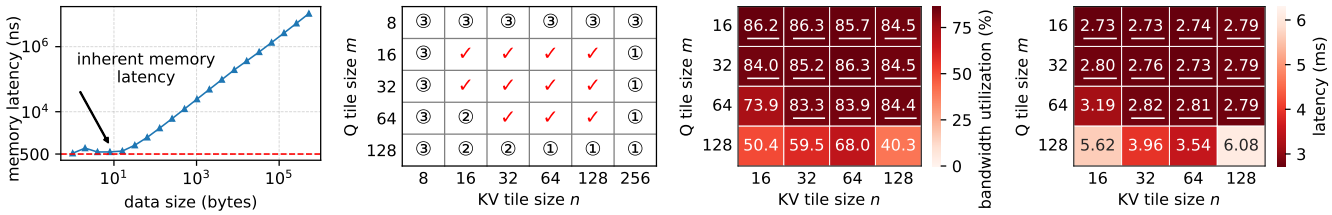
Lazy Update. Although the pack scheduler is linear, its overhead can grow with batch size and number of KV blocks. We mitigate this with a lazy-update strategy that (1) reuses a scheduling result across continuous-batching iterations until the block table changes (*e.g.,* request arrivals/departures or new KV block assignments) and (2) moves the scheduler into the serving system and runs it asynchronously once the block table is available. These reduce scheduler invocations from once per transformer layer to once several continuous-batching iterations and overlap scheduling with pre-attention task (*e.g.,* metadata preparation, QKV projection), thereby substantially reducing the exposed scheduling latency (see §8.7) without affecting model accuracy.

5.2 Multi-tile Kernel

Insight and Approach. Given packed CTAs, PAT must choose per-CTA Q-tile m and KV-tile n to maximize resource efficiency. Because query sizes and KV lengths vary widely, single–tile-size kernels with padding (*e.g.,* prior works [8, 33, 50, 60]) cannot adapt and suffer poor utilization (§3.3). To better exploit GPU shared memory and registers, we introduce a *customized tiled attention kernel design* that consists of:

[2] To ensure numerical accuracy, intermediate results are stored in FP32, so the overhead is multiplied by 2.

[3] V and E denote numbers of nodes and edges in tree-structured block table.



(a) memory latency **(b)** tile size configurations **(c)** bandwidth equivalence **(d)** latency equivalence

Figure 8. Multi-tile kernel design and validation on A100-SXM4-80GB GPU. (a) Global-to-shared memory transfer latency for varying data sizes (averaged over 1,000 reads). (b) Offline-selected tile-size configurations; check marks denote feasible settings, and circled numbers indicate violated constraints. (c) Average bandwidth utilization under different tile sizes. (d) Average kernel latency under different tile sizes.

Algorithm 1: TreeHeuristic

Input: Root node *root*, corresponding KV *blocks*
Output: Packs $\mathcal{P}$ from the tree

$\mathcal{P} \leftarrow \emptyset$;
if IsLeaf(r) **then**
  // Pack the non-shared KV into a CTA
  return Pack$(root.\text{query}, blocks)$;
for $c \in$ Children$(root)$ **do**
  // Use profit model to choose the scheme
  if $4 \times c.size < root.len_s$ **then**
    // Scheme 1: split root and child into separate CTAs
    $\mathcal{P} \leftarrow \mathcal{P} \cup$ TreeHeuristic$(c, c.\text{blocks})$;
  else
    // Scheme 2: merge root's blocks with c's blocks
    $\mathcal{P} \leftarrow \mathcal{P} \cup$ TreeHeuristic$(c, c.\text{blocks} \cup blocks)$;
    $root$.RemoveQuery$(c.\text{queries})$;
// Pack remaining queries and KV blocks into a CTA
$\mathcal{P} \leftarrow \mathcal{P} \cup$ Pack$(root.\text{queries}, blocks)$;
return $\mathcal{P}$;

(1) a multi-tile kernel suite, where feasible (m, n) configurations are derived from offline hardware and CTA-constraint analysis and implemented as resource-efficient kernels; and (2) a tile-size selector, an online decision-tree that selects the per-CTA (m, n) to balance performance and parallelism.

Multi-tile kernel. Tile sizes (Q tile m and KV tile n) critically affect Tensor Core efficiency: they determine a CTA's shared-memory and register demand, which in turn constrain resident CTA concurrency and active warps. To obtain resource-efficient kernels, we derive three key constraints that significantly reduce the (m, n) search space.

① ***Register and shared-memory constraints*** (upper bounds on m, n). Increasing m or n raises CTA shared-memory and register usage and can induce register spilling [26]. To keep the kernel within hardware limits, we enforce two bounds: (1) *Shared-memory constraint.* One CTA's shared-memory usage comprises the Q tile, K/V tile, and intermediate results (data type size b', usually higher precision). It must not exceed per-SM shared memory S_{smem}:

$$m\,h\,b \;+\; n\,h\,b \;+\; m\,h\,b' \;\le\; S_{\text{smem}}.$$

(2) *Register constraint.* We bound per-thread[4] and aggregate register use to avoid spilling: per-thread registers $\le S_{\text{reg\_thr}}$; total registers of concurrent CTAs on an SM $\le S_{\text{register}}$. Because compiler effects make analytic estimates unreliable, we obtain per-thread $R_{\text{thr}}(m, n)$ and per-CTA $R_{\text{CTA}}(m, n)$ via offline compilation and static analysis, and enforce:

$$R_{\text{thr}}(m, n) \le S_{\text{reg\_thr}}, \qquad C \cdot R_{\text{CTA}}(m, n) \le S_{\text{register}}.$$

② ***High bandwidth utilization*** (lower bound for n). To saturate global memory bandwidth, the pipeline must *keep enough data in flight* to cover the inherent memory latency. Figure 8a shows transfer latency vs. data size: the flat region gives the inherent latency L (ns) and the linear region the sustainable bandwidth B (Bytes/ns). Thus the in-flight data D_{flight} must satisfy $D_{\text{flight}} \ge L \times B$. In the attention pipeline D_{flight} is the total size of all K or V tiles being loaded by concurrently resident CTAs, *i.e.,* $D_{\text{flight}} = S\,C\,n\,h\,b$ where S is the number of SMs, C the concurrent CTAs per SM, h the head dimension, and b the KV datatype size (bytes). Rearranging gives the lower bound, which complements the CUTLASS-derived constraints and ensures the memory bus remains well utilized:

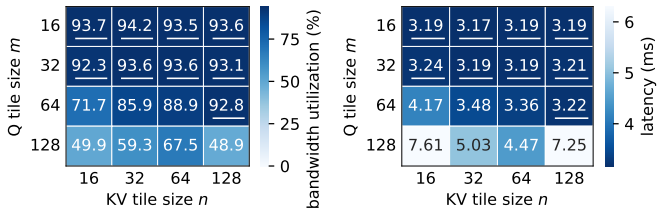
$$n \;\ge\; \left\lceil \frac{L\,B}{S\,C\,h\,b} \right\rceil,$$

③ ***CUTLASS constraint*** (lower bounds for m and n): Efficient use of CUTLASS/CuTe MMA requires both tile sizes to be powers of two and at least[5] 16 [27]:

$$m, n \in \{2^k \mid k \in \mathbb{N},\ 2^n \ge 16\}$$

Put it together. Based on ① to ③, an offline configuration solver is designed to compute feasible tile size (m, n) pairs per hardware target, thereby providing kernels that execute efficiently for dynamic CTAs. Figure 8b shows a set of available tile size configuration under A100-80GB.

[4]Commonly 255 32-bit registers per thread on recent GPUs.
[5]It depends on data format; e.g., the minimum tile size is 32 for int8.



(a) bandwidth equivalence **(b)** latency equivalence

Figure 9. Multi-tile kernel validation on H100-SXM4-80GB GPU. (a) Average bandwidth utilization under different tile sizes. (b) Average kernel latency under different tile sizes.

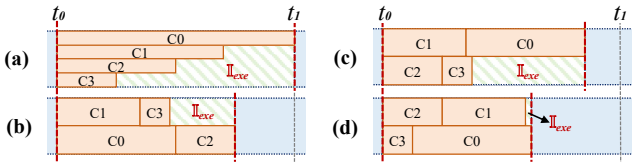


Figure 10. Execution pipeline of four CTAs under different concurrency. (a) High concurrency with dynamic KV lengths yields low tail efficiency and a large execution bubble $\mathbb{I}_{exe}$. (b–d) Lower concurrency cuts per-CTA latency and thus reduces $\mathbb{I}_{exe}$ across the shown execution orders.

Tile Selector. Given the packed CTAs and feasible multi-tile kernel configurations, the tile selector assigns a tile size configuration pair (m, n) to each CTA at runtime using a set of rules. These rules are derived offline based on analysis of (1) when different tiles behave equivalently, and (2) how m and n affect resource efficiency.

Kernel equivalence. Per the offline constraint solving above, the candidate (m, n) configurations sustain high bandwidth utilization, and decode-attention latency is dominated by global memory bandwidth. Thus, for decode batches *without* shared prefixes or execution bubbles, these configurations are *performance-equivalent* in bandwidth and latency. To validate this, we executed a decode batch without prefixes (KV length 1024) under various configurations and used batch size 1134, which is a common multiple of CTA concurrency across configurations on A100, avoiding execution bubbles. As shown in Figure 8c and Figure 8d, all candidate configurations (underlined) sustain 83%-86% bandwidth utilization and exhibit similar end-to-end latency (difference $< 2\%$). This demonstrates that, in the absence of prefixes and bubbles, varying the tile configuration within the feasible set does not change CTA performance.

Porting PAT's multi-tile kernel to other GPUs only requires re-deriving equivalent tile size configurations using constraints ①-③. On Hopper H100-SXM-80GB GPUs, this procedure removes the $(64, 32)$ and $(64, 64)$ configurations from Figure 8b, and the remaining entries form the equivalent kernel set. Figure 9 reports validation on H100 at batch size 1188, a common multiple of CTA concurrency across all configurations[6]. All equivalent configurations achieve 92.3%-94.2% bandwidth utilization and similar kernel latency. These results indicate that the same constraint-based procedure generalizes across architectures.

Deriving Q tile m. The shared prefixes make the query size per CTA dynamic. To mitigate padding-induced memory waste $\mathbb{I}_{mem}$ on the m dimension, the selector uses a *round-up* rule: given CTA's query size q, choose the smallest m in feasible q tile sizes with $m \geq q$. For instance, when $q = 20$, it chooses $m = 32$ rather than 16, since $m = 16$ will split the query into two CTAs and result in redundant accesses of the shared KV cache. Larger and performance-equivalent tile sizes such as 64 or 128 are also avoided to preserve on-chip memory for KV tile size n selection as follows.

Deriving KV tile n. The choice of n must adapt to KV length. *For long KV*, high CTA concurrency causes severe execution bubbles and poor tail efficiency (Figure 10a), so we prefer larger n: this raises per-CTA on-chip memory, reduces per-SM concurrency C, thereby increasing bandwidth available per CTA and shrinking the execution bubble $\mathbb{I}_{exe}$ across different execution orders (Figure 10b-d). *For short KV*, the last tiling iterations can make the compute-only portion significant (right side of Figure 3b), so a smaller n shortens the final tile and avoids a compute bubble is preferred. *e.g.,* at KV length 192, n=128 yields $(192 - 128)/128 \approx 50\%$ compute bubble in the last tile, while n=64 removes it and is faster due to kernel equivalence. Guided by these trade-offs, we profile each candidate n offline by sweeping KV length to derive the largest performance-equivalent tile sizes until the choice stabilizes; the resulting mapping is encoded as a piecewise decision tree. During online serving, tile selector performs a constant-time lookup per CTA to choose the profiled n.

6 Multi Kernel Forward

After §5 packs the batch into CTAs and selects multi-tile kernels, we design two forward-stage strategies to mitigate kernel execution bubbles $\mathbb{I}_{\text{exe}}$ (§3.3).

Multi-Stream Forward. Kernels with different tile size configurations launch and execute sequentially on the GPU, since the GPU requires static kernel launch parameters derived from tile size. This serial execution incurs resource inefficiencies due to frequent kernel launches and execution bubbles, with the latter accumulating across consecutive kernels. To address this, we create a separate CUDA stream for each distinct tile size configuration (m, n) obtained from §5.2. The scheduler groups CTAs by their (m, n) pair and enqueues each group into its corresponding stream, so that all CTAs with the same configuration execute sequentially within that stream, while different streams run in parallel.

[6]Batch size 1134 for A100 and 1188 for H100 are only used in the kernel-equivalence validation, not in the evaluation.

This design overlaps the launch overhead of subsequent kernels with the execution of preceding kernels and mitigates execution bubbles by kernel parallelism (§8.6).

Long KV Split. Multi-stream execution alone cannot eliminate execution bubbles in all cases, because some CTAs may have KV lengths that are orders of magnitude larger than others. We therefore adopt a KV-dimension splitting strategy similar to [33]. Specifically, we split any CTAs with KV length exceeding the mean KV length for all CTAs of the batch into equal parts to keep the KV length under the mean value. This shortens the completion time of the last finishing CTAs and improves overall SM utilization of kernel execution.

7 Output Merge

We implement a lightweight merge kernel that uses online softmax [9] to combine partial results at per-query granularity. Each CTA produces three per-query and per-head intermediates: a max score, a log-sum-exp accumulator, and a partial value-weighted sum. The merge kernel loads these intermediates from global memory, reduces them with online softmax, normalizes the accumulated value-weighted sum, concatenates all heads, and writes the final query output back to global memory. The small global read/write overhead for these intermediates is accounted for in the pack scheduler's overhead analysis when deriving the end-to-end packing scheme (5.1).

8 Evaluation

In this section, we present the implementation of PAT (§8.1) and the experimental setup (§8.2). We then conduct a set of experiments to answer three key questions:

1. How does PAT 's efficiency scale across diverse batch sizes, models, and prefix structures? (§8.3)
2. What performance improvement does PAT achieve for online serving under real-world workloads? (§8.4, §8.5)
3. What is the contribution of each feature of PAT to the overall performance gain? (§8.6)
4. What is the impact of PAT's overhead? (§8.7)

8.1 Implementation of PAT

We implement PAT as a full attention kernel for the decode stage with about 3k lines of Cutlass/CuTe [29, 30] and C++ code. The multi-tile kernel (§5.2), multi-stream forward (§6), and merge kernel (§7) are built with Cutlass/CuTe. The asynchronous pack scheduler (§5.1) and API wrappers are implemented in C++. To overlap the data transfers with computation, all data movement from the global memory to the shared memory uses the `cp_async` primitive, together with double buffering [26]. We expose the kernel API to Python via pybind11, and integrate it into vLLM [16] (v0.9.0) as an off-the-shelf plugin with about 1.2k lines of Python code. PAT treats vLLM's paged KV cache as its substrate: KV entries are managed as fixed-size blocks in block tables, and the pack scheduler operates only on block IDs produced by vLLM. This design lets PAT reuse vLLM's existing KV paging implementation. To enable PAT in vLLM, only an environment variable `VLLM_ATTENTION_BACKEND=PAT` is required.

8.2 Experiment Setup

Models and Testbed. For kernel benchmark (§8.3), we evaluate PAT on both NVIDIA A100 GPU (80GB) and NVIDIA H100 GPU. For end-to-end online serving (§8.4), we use two representative LLMs, Qwen3-8B and Llama-3-8B, on a single A100 GPU. We further evaluate PAT under distributed settings and Mixture-of-Experts (MoE) architectures in §8.5 using Qwen2.5-72B-Instruct on four A100 GPUs and Qwen3-30B-A3B on one A100 GPU. The software environment is CUDA 12.4 and PyTorch 2.7.0.

Baselines. We compare against seven attention implementations spanning query-centric and KV-centric designs:

1. `FlashAttention` [8, 9] (v2.5.9): query-centric; maps each query to a CTA with a fixed tile size config $(64, 128)$.
2. `FlashInfer` [50] (v0.2.5): query-centric; improves SM load balance by dynamic CTA partitioning; decoding tile config $(16, 128)$.
3. `FastTree` [33]: KV-centric; uses a compute-oriented cost model to pack and reduce repeated KV loads; two tile configs $(64, 32)$ and $(16, 32)$.
4. `RelayAttention` [60]: KV-centric; packs first-level shared prefixes into CTAs to cut redundant memory accesses and runs them with FlashAttention's kernel.
5. `RelayAttention++`: our extension of RelayAttention to exploit vLLM-style KV-cache reuse; it stores shared KV blocks from non-first-level prefixes in the same physical space so redundant KV loads can benefit from L2 cache, further improving performance (§8.3).
6. `DeFT` [47]: KV-centric, aggregates queries with shared KV and adjusts the KV length in each CTA for load balance; fixed tile size config $(32, 16)$.
7. `Cascade Inference` [51]: KV-centric, packs prefixes into CTAs using fixed settings.

Kernel-Performance Workloads (§8.3). To compare kernel performance under identical batch and prefix structures, we construct *synthetic decode batches* as input, following [33]. Each decode batch is specified by B and L. B defines the prefix-tree structure and the number of leaves (*i.e.,* batch size). *e.g.,* $B = [1, 4, 16]$ yields two shared-prefix levels with 1 and 4 nodes, and 16 leaves. L gives KV lengths per level. *e.g.,* $L = [128, 256, 1024]$ sets level-1 and level-2 shared prefixes to 128 and 256 tokens, with 1024 non-shared tokens per request. We vary (B, L) combinations to reflect different shared-prefix structures and batch settings as in Figure 11. Besides, we choose *four head configurations* (#heads, #kv_heads) common in Llama [3], Qwen [46], and Gemma [40] models:

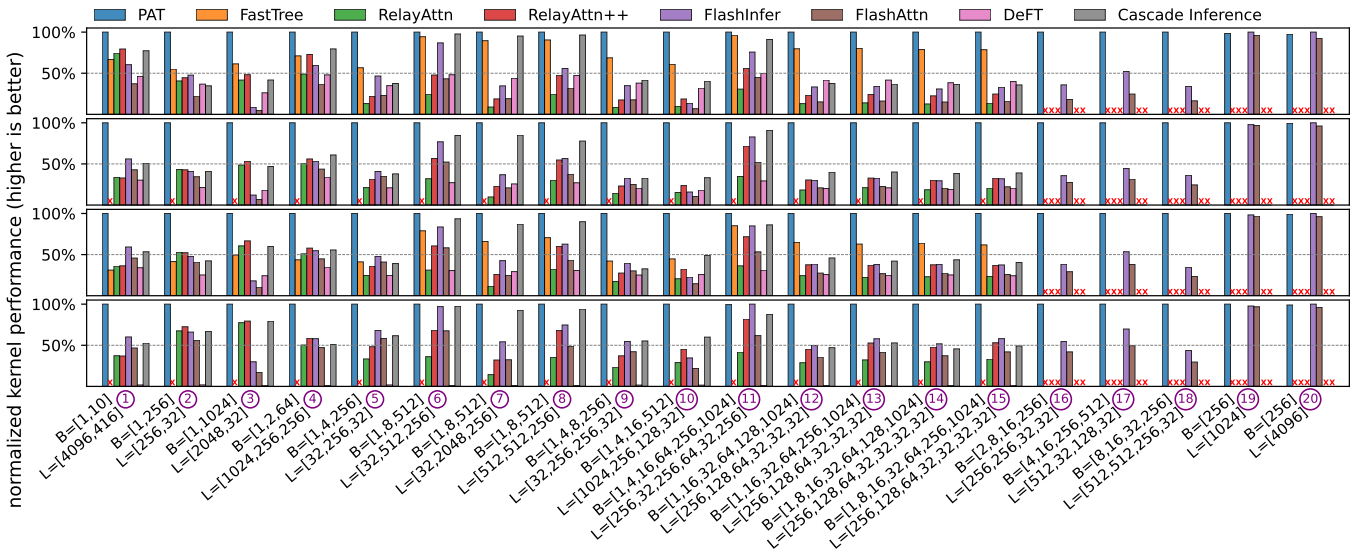


Figure 11. Normalized kernel performance (higher is better) of PAT and the baselines for the attention computation across various decode batch configurations on NVIDIA A100 GPU (80GB). The four panels from top to bottom show head configurations (*num_attention_heads*/*num_key_value_heads*) of 32/32, 16/8, 32/8, and 64/8. Missing bars arise because RelayAttention lacks support for multi-level or multiple first-level prefixes, and FastTree does not support the 16/8 and 64/8 head settings.

(64, 8), (32, 8), (16, 8), and (32, 32). The head dimension and data type are set to the commonly used 128 and FP16.

End-to-end workloads (§8.4 and §8.6). We evaluate PAT under online serving using vLLM [16] (v0.9.0) on two real-world traces. (1) `toolagent` [34]: tool/agent workloads with task-specific system prompts (overall cache hit rate 59%). (2) `conversation`: combines the Meta-AI system instruction (total lengths 2517/2522 tokens for Llama3/Qwen3 tokenizers) with burstgpt prompts, following prior work [33]. We randomize language and country fields in the system instruction to create a three-level prefix whose lengths are 46, 348, 2123 (Llama3) or 45, 351, 2126 (Qwen3). We use only the first 30 minutes of each trace due to cost limit.

Metrics. For kernel performance, we primarily compare attention latency under varied input configurations. For each configuration, we run 20 repetitions and report the average completion latency. For end-to-end comparison, we focus on three metrics: average request completion latency, time to the first token (TTFT), and time per output token (TPOT).

8.3 Kernel Performance

Overall results. Figure 11 reports normalized kernel performance (metric latency^{-1}, normalized to PAT) across decode-batch configurations on NVIDIA A100 GPU (80GB)[7]. For configurations with shared prefixes (①–⑱), PAT achieves up to 21.5×, 11.7×, 3.2×, 11.9× and 5.7× speedups over FlashAttention, FlashInfer, FastTree, RelayAttention and RelayAttention++, respectively, across four attention-head settings. These gains arise from three factors: (1) a prefix-aware packing scheduler that cuts redundant global KV accesses (§5.1); (2) a multi-tile kernel combined with an online tile-size selector that adapts (m, n) per CTA to better use bandwidth and on-chip memory (§5.2); and (3) multi-stream forward execution and long-KV splitting that mitigate execution bubbles caused by multi-tile kernels and KV-length dynamics (§6). Together, these designs make PAT consistently more efficient than the baselines.

Compared with query-centric kernels. Against query-centric FlashAttention and FlashInfer, PAT reduces attention latency by 67.8% and 52.1% on average under configurations with prefixes (①–⑱). The gap widens with larger batch size or longer shared prefixes (*e.g.,* ④–⑤ and ⑥–⑦) because the one-query-per-CTA design of query-centric kernels forces repeated global KV loads that grow more costly in those cases. We also include configurations ⑲ and ⑳ in Figure 11, which remove shared prefixes and thus represent workloads without prefix reuse. In this case, PAT no longer reduces global memory accesses but still benefits from its multi-tile kernel and multi-stream forward. As a result, PAT achieves 1.6% lower latency on average with fewer execution bubbles.

Compared with KV-centric kernels. FastTree, which also targets multi-level shared prefixes, remains the strongest baseline but is still 3.8%–68.9% slower than PAT for two reasons: (1) its compute-oriented packing cost model is ill-suited to memory-bound decode attention (§2.3 and §3.2); and (2) its double-tile approach launches two kernels serially as in Figure 15b, introducing execution bubbles. By contrast, PAT uses a memory-oriented packing strategy plus multi-stream

[7]Evaluation results on NVIDIA H100 GPU are presented in Appendix A.

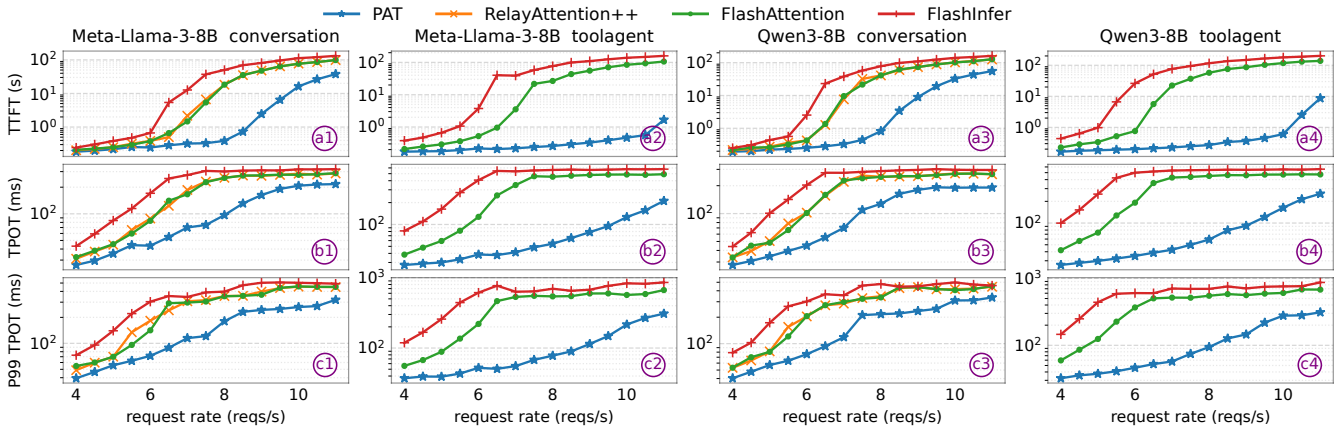


Figure 12. End-to-end performance of PAT and the baselines under two models and two traces. Note that RelayAttention++ lacks support for multiple first-level prefixes, so its results on `toolagent` trace are unavailable.

forward to avoid these inefficiencies. RelayAttention++ cuts latency by 67.4% versus RelayAttention, confirming that L2 plus KV-cache reuse reduces redundant global loads, yet RelayAttention++ is still 1.7× slower than PAT, showing that L2 cache alone cannot fully eliminate redundant KV loads. In addition, DeFT and Cascade Inference both use a naive packing scheme for shared prefixes. While DeFT's load-balancing reduces SM execution bubbles from long-tail CTAs, neither method effectively reduces global memory accesses, which dominate decode attention latency. Therefore, PAT achieves 76.6% and 41.2% lower attention latency on average than DeFT and Cascade Inference, respectively.

8.4 End-to-End Comparison

Overall trends. We compare PAT with three baselines across two models (Llama-3-8B and Qwen3-8B) and two real-world traces (conversation and toolagent) as in §8.2. Figure 12 shows the trend of mean TTFT, mean TPOT, and P99 TPOT as we vary the request rate. Specifically, as in subfigures (b1) to (b4), PAT reduces mean TPOT at the same request rate by 17.2–68.1% over RelayAttention++, 17.0–89.5% over FlashAttention, and 32.2–93.1% over FlashInfer. Furthermore, as shown by the first and the third rows of Figure 12, the TPOT reduction allows PAT to finish incoming requests faster at the same request rate. This yields 9.3–98.6%, 10.1–99.6%, and 22.5–99.8% lower TTFT than the three baselines.

Scaling with request rate. The performance gap between PAT and the baselines first widens and then slightly contracts as the request rate grows. This is because: (1) Higher request rate forms larger decode batches under continuous batching, which increases the redundant global memory accesses and exposes more opportunity for PAT; (2) When the batch size further increases, the number of CTAs and the overall attention runtime grow, so that the execution bubble becomes smaller; the performance improvement of multi-stream forward and long-KV split therefore shrinks.

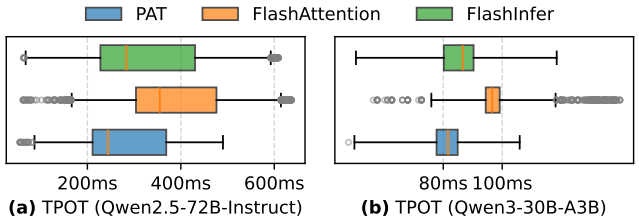
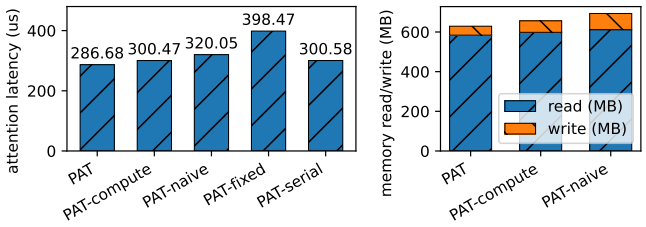


Figure 13. End-to-end performance of PAT and baselines under TP/PP and MoE architectures.

Why baselines fail. PAT achieves significant gains for its prefix-aware pack scheduler, multi-tile kernel, and multi-stream forward. In contrast, RelayAttention++ supports only a single-level system prefix and delegates the forward kernel to FlashAttention, so its curves largely track FlashAttention, which can not mitigate redundant KV loads at all. Model configuration also matters. Llama-3-8B's 8K context length limit (vs. 32K for Qwen3-8B) requires us to filter ultra-long requests, which reduces the requests with long non-prefix context and leads to lower absolute latency for both RelayAttention++ and PAT on Llama-3-8B compared with that of Qwen3-8B. Besides, FlashInfer improves SM utilization via long-CTA-splitting load balance, which helps at low request rate while adding scheduling overhead that grows with request rate, resulting in the highest TPOT.

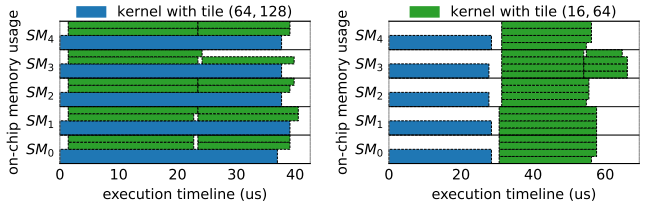
8.5 Distributed and MoE Extensions

Recent production LLM deployments increasingly rely on multi-GPU inference as model weights exceed a single GPU's memory capacity, and many models adopt Mixture-of-Experts (MoE) architectures to increase capacity at moderate cost. PAT seamlessly supports tensor parallelism (TP), pipeline parallelism (PP), and MoE: TP and PP partition attention heads or transformer blocks across GPUs, while each device retains full KV cache for its assigned heads or blocks; MoE



(a) attention latency **(b)** memory read/write

Figure 14. Results of ablation baselines. (a) Average attention latency. (b) Global memory read and write size.



(a) multi-stream **(b)** serial execution

Figure 15. CTA execution pipeline on $SM_0 - SM_5$ with two tile size configurations (collected by PTX [24]), where white space represents execution bubbles. (a) Multi-stream execution pipeline. (b) Serial execution pipeline.

architecture replaces the FFN layer with multiple experts but leaves attention execution unchanged. Therefore, PAT's pack scheduler and multi-tile kernel run unchanged.

To validate these properties, we evaluate PAT under the toolagent trace using Qwen2.5-72B-Instruct with TP=2 and PP=2 on four A100 GPUs, and Qwen3-30B-A3B on one A100 GPU. As shown in Figure 13, compared to three baselines, PAT reduces average TPOT by 14.3 – 26.7% on Qwen2.5-72B-Instruct and by 5.53 – 16.9% on Qwen3-30B-A3B. These results suggest that prefix-aware packing and the multi-tile kernel remain effective under common distributed and MoE deployment configurations.

8.6 Ablation Study

To evaluate the contribution of each design in PAT, we build the following ablation baselines: (1) PAT-compute, which adopts the cost model from FastTree [33] for the packing scheduler; (2) PAT-naive, which simply packs each node in the tree-structured block table into a CTA; (3) PAT-fixed, which disables the multi-tile kernel of PAT and instead uses the fixed tile configuration (64, 128) as in FlashAttention [36]; (4) PAT-serial, which disables the multi-stream forward mechanism of PAT and adopts the serial multi-kernel execution similar to FastTree [33]. We use the same synthetic traces as in §8.3 and adopt the attention head configuration of Llama-3-8B to compare PAT with ablation baselines.

Effectiveness of the pack scheduler. As shown in Figure 14a, the average attention latency of PAT-compute and PAT-naive is higher than PAT by 4.6% and 10.4%, respectively. PAT-compute adopts a compute-oriented cost model that selects the packing scheme with the minimum computation, which contradicts the memory-bound nature of decode attention. Meanwhile, PAT-naive only considers the benefit of packing but ignores the additional intermediate reads and writes, leading to high overhead. As shown in Figure 14b, their average memory read/write is higher than PAT by 10.9% and 16.7%, respectively. This confirms the rationality of PAT 's memory-oriented cost model and heuristic pack scheduler.

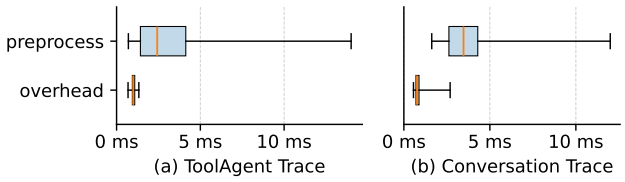


Figure 16. Overhead of pack scheduler and pre-attention task latency of serving system.

Importance of multi-tile kernel. As shown in Figure 14a, enforcing a fixed tile size (PAT-fixed) increases attention latency by 39% compared to PAT. In the ablation workload, CTAs exhibit query sizes from 1 to 64 and KV lengths from 32 to 4096, which makes one-size-fits-all kernels highly inefficient. Prior kernels [36, 47, 50, 60] adopt such fixed designs, leading to padding overhead and execution bubbles. In contrast, PAT's multi-tile kernel enables efficient adaptation to diverse CTA configurations.

Effectiveness of multi-stream forward. As shown in Figure 14a, the average attention latency of PAT-serial is 4.8% higher than PAT. This is because serial execution aggravates execution bubbles. For example, in a two-level prefix decode batch (Figure 11 ⑥), Figure 15a and Figure 15b show the CTA execution pipelines of PAT and PAT-serial, where PAT-serial suffers from substantial memory waste and execution bubbles. At runtime, PAT's multi-tile kernel selects a tile configuration for each CTA from the equivalent configuration set. In our A100 experiments, each batch typically uses 1–5 of the 11 available configurations (Figure 8). PAT's multi-stream forward then launches a separate CUDA stream for CTAs with each active tile configuration, allowing these streams to run in parallel under the GPU scheduler. With multiple configurations, some launch bubbles remain, as indicated by the blank region on the left of Figure 15a. Nevertheless, parallel streams substantially reduce execution bubbles compared to serial execution, thereby lowering attention latency.

8.7 Overhead Analysis

The primary overhead introduced by PAT comes from the pack scheduler, which packs decode batches into CTAs at runtime. PAT's lazy update mechanism (§5.1) is designed to mitigate the overhead. It reduces the scheduler's triggering

frequency without affecting correctness and overlaps its execution with the serving system's pre-attention tasks (*e.g.,* metadata preparation, QKV projection). To validate, we measure both the scheduling latency and the pre-attention task latency under toolagent and conversation traces at request rates of 5 and 8 req/s. As shown in Figure 16, the average scheduling latency is lower than the pre-attention task latency by 42.3% and 49.6%, respectively. Consistently, the P99 TPOT in Figure 12 ⓒ1-ⓒ4 shows 19.4–93.4% reduction compared with the baselines. Therefore, when running on an asynchronous CPU thread, the pack scheduler will *not* introduce additional end-to-end latency, demonstrating the effectiveness of the lazy update mechanism in PAT.

9 Discussion

Prospects and Limitations. PAT leverages cross-request prefix sharing to improve bandwidth utilization and cut redundant global memory accesses in memory-bound decode attention. Its effectiveness depends on three factors: (1) *hardware compute-to-bandwidth ratio.* As GPUs become more compute-dominant (*e.g.,* NVIDIA V100 to B200: 139 to 312 FLOP/Byte), the gap between compute and memory widens, so memory-focused designs like PAT become increasingly valuable; (2) *model architecture.* PAT yields large gains for common architectures that retain a KV cache (MHA, GQA), but benefits may shrink for architectures that compress or remove KV state (MLA [21], linear attention [37], MLKV [61]); and (3) *prefix-sharing ratio in the batch.* High concurrency with cross-request shared prefixes amplifies PAT 's advantage, whereas small batches or workloads without shared prefixes limit the improvement (see Figure 11 ①, ⑲–⑳).

Gap to Optimal. PAT reduces memory waste and execution bubbles, guided by hardware and pipeline analysis. However, GPU scheduling is uncontrollable, leaving residual bubbles (Figure 15a) and a gap from the theoretical optimum. Yet PAT consistently outperforms state-of-the-art baselines.

10 Related Work

KV Cache Related Optimization. Prior work reduced KV cache memory and fragmentation: FasterTransformer [28] provided an early static-batching implementation, Orca [52] improved utilization with continuous batching but relied on pre-allocated caches, which leads to memory fragmentation. vLLM introduced a paged KV cache to cut waste by virtual memory management [16]. SGLang's prefix-reuse lets shared prefix KV blocks be reused across requests and is widely used in production [55]. Recent systems [11, 17, 35] further extend KV capacity by offloading KV blocks between GPU and CPU or NVMe storage, which reduces on-GPU memory pressure at the cost of additional data movement. These approaches shrink KV memory costs but do not accelerate decode attention, which dominates latency as context and output lengths grow. PAT is orthogonal to these approaches because its pack scheduler only relies on logical KV block IDs, and the serving system transfers the required KV blocks to GPU memory before the attention kernel executes.

Attention Kernel Optimization. Fused, on-chip attention kernel implementations like FlashAttention and FlashInfer reduce global memory traffic by combining attention steps into a single kernel [9, 36, 50], but their query-centric (one-query-per-CTA) design cannot exploit workload-level prefix sharing. Subsequent works [19, 49, 56, 60] pack the single shared system prompt to reduce redundant memory accesses, but can not generalize to workloads with multi-level prefixes. Several recent works [14, 47, 51] extend to multi-level prefixes but rely on simple packing strategies that overlook trade-offs between overhead and savings. [33] also addresses multi-level prefix optimization, but its compute-oriented packing cost model mismatches the memory-bound nature of decode attention, leading to suboptimal results. Furthermore, all these works use one-size-fits-all kernel design, leading to resource inefficiency. In contrast, PAT uses a memory-centric packing strategy with multi-tile kernels and multi-stream execution to reduce redundant memory accesses and improve resource utilization for dynamic workloads.

GPU Scheduling. GPU scheduling strategies further improve kernel efficiency by balancing compute and memory demands: [15] fuses prefill and decode via virtual CTAs to expose resource complementarity, and [59] groups SMs so different SMs specialize in specific tasks, raising utilization. These techniques are largely orthogonal to PAT and could be combined to further reduce execution inefficiencies. Work like [38] shows that CTA-style scheduling on alternative accelerators (*e.g.,* NPUs) can help eliminate the remaining resource bubbles in PAT 's execution (§9). Here, we focus on GPU decode-attention optimizations and leave heterogeneous-scheduler integration to future work.

11 Conclusion

In this work, we present PAT, a prefix-aware attention kernel implementation for LLM decoding. PAT reduces redundant global memory accesses and improves bandwidth utilization through a pack–forward–merge paradigm. The design incorporates a heuristic pack scheduler, a resource-efficient multi-tile kernel, and forward-stage optimizations that mitigate execution bubbles. We implement PAT as an off-the-shelf plugin to vLLM and demonstrate that it reduces the attention latency by 53.5% on average under synthetic workloads and reduces the TPOT by up to 93.1% under real-world workloads compared with existing works.

12 Acknowledgments

We appreciate the insightful feedback from the anonymous reviewers and our shepherd, Seonjin Na. This work is supported by the National Natural Science Foundation of China under grants No. 62572341 and No. 62202328.

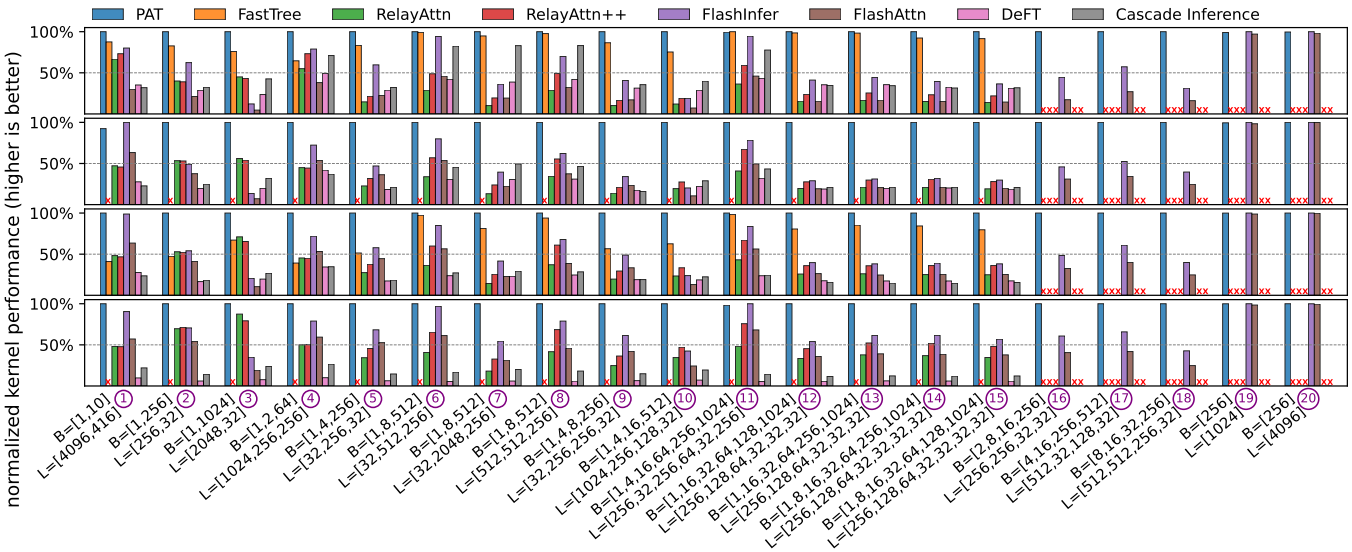


Figure 17. Normalized kernel performance (higher is better) of PAT and the baselines for the attention computation across various decode batch configurations on NVIDIA H100 GPU. The four panels from top to bottom show head configurations (*num_attention_heads*/*num_key_value_heads*) of 32/32, 16/8, 32/8, and 64/8. Missing bars arise because RelayAttention lacks support for multi-level or multiple first-level prefixes, and FastTree does not support the 16/8 and 64/8 head settings.

A Kernel Performance on H100 GPU

PAT's multi-tile kernel adapts to different GPU architectures by re-deriving equivalent tile-size configurations from the three constraints in §5.2. This allows the kernel to match different memory hierarchies and bandwidths. To validate this, we compare PAT with the baselines in §8.2 on an NVIDIA H100 GPU, using configurations in Figure 11. As in Figure 17, PAT achieves kernel performance of 1.3 – 6.9× to baselines with shared prefixes (①-⑱), and achieves consistent performance without shared prefixes (⑲ and ⑳). These results further confirm the robustness of PAT's design.

B Artifact Appendix

B.1 Abstract

We provide the source code of PAT along with scripts to reproduce experimental results presented in §8. This appendix includes instructions for reproducing the kernel performance evaluation results in Figure 11 and four representative end-to-end performance evaluation results in Figure 12.

To expedite artifact evaluation, a pre-built Docker image is available that contains the fully configured environment, pre-compiled source code, and datasets as specified in §8.2. The experiment require an x86-64 Linux host with at least 64GB RAM, 200GB of free disk space, and an NVIDIA A100 GPU (80GB). For convenience and consistent performance, we recommend using a Google Cloud a2-ultragpu-1g instance with the "Deep Learning VM with CUDA 12.4" system image.

B.2 Artifact check-list (meta-information)

- **Algorithm:** Prefix-aware attention kernel implementation.
- **Compilation:** Pre-compiled with in a Docker image.
- **Model:** Llama3-8B and Qwen3-8B.
- **Run-time environment:** Docker, NVIDIA Container Toolkit, and CUDA driver ≥ 550.
- **Hardware:** 1× NVIDIA A100-80GB GPU.
- **Metrics:** kernel latency, mean TTFT, mean TPOT, P99 TPOT.
- **Output:** Figure and console.
- **Experiments:** Kernel performance (Figure 11); End-to-end performance (Figure 12).
- **How much disk space required (approximately)?:** 200GB.
- **How much time is needed to prepare workflow (approximately)?:** 30 minutes.
- **How much time is needed to complete experiments (approximately)?:** ≈2 hours for kernel performance experiment (Figure 11); ≈10 hours for the selected end-to-end performance experiments (Figure 12).
- **Publicly available?:** Yes.
- **Code licenses (if publicly available)?:** MIT.
- **Archived (provide DOI)?:** 10.5281/zenodo.18217189.

B.3 Description

B.3.1 How to access.

- GitHub: https://github.com/flashserve/PAT
- Zenodo: 10.5281/zenodo.18217189
- Pre-complied Docker image: flashserve/pat:ae

B.3.2 Hardware dependencies. Requires an x86-64 Linux host with at least 64GB of RAM, 200GB of free disk space, and an NVIDIA A100 GPU (80GB). For convenience and consistent performance, we recommend using a Google Cloud a2-ultragpu-1g instance with the "Deep Learning VM with CUDA 12.4" system image.

B.3.3 Software dependencies. Docker, NVIDIA Container Toolkit, and NVIDIA driver ≥550 are required. Other software has been installed within the provided Docker image.

B.3.4 Data sets. The data sets used in experiments are listed in §8.2 and are pre-downloaded in the Docker image.

B.3.5 Models. The model used in these experiments are listed in §8.2, including Qwen3-8B and Llama-3-8B.

B.4 Installation

1. Clone the GitHub repository.

```
$ git clone
  https://github.com/flashserve/PAT.git
```

2. Pull the pre-built Docker image.

```
$ # about 50GB, including model weights
$ docker pull flashserve/pat:ae
```

3. Start a Docker container with GPU access and mount the repository.

```
$ docker run -it --gpus all --shm-size=64g \
    -v ${PWD}/PAT:/workspace/PAT \
    -w /workspace/PAT \
    flashserve/pat:ae /bin/bash
```

B.5 Experiment workflow

1. Run the kernel performance experiments. This experiment takes about 1.5 hours to complete.

```
$ cd /workspace/PAT/benchmark
$ bash ./run_kernel_bench.sh
```

2. Run the end-to-end serving performance experiments. Note that completing all experiments requires over 60 GPU-hours, so we provide two scripts for convenience: (1) `run_e2e_bench_part.sh`: runs a subset of experiments (QPS=7&9, all workloads, all baselines) for quick verification; (2) `run_e2e_bench_full.sh`: runs all experiments in Figure 12.

```
$ cd /workspace/PAT/benchmark
$ # Quick verification (8-10 GPU-hours)
$ bash ./run_e2e_bench_part.sh
$ # Full experiments (over 60 GPU-hours)
$ # bash ./run_e2e_bench_full.sh
```

B.6 Evaluation and expected results

Generate plots.

```
$ cd /workspace/PAT/plot
$ python eval_kernel_perf.py --log-file \
  ../benchmark/kernel_perf.json
$ python eval_e2e_from_jsonl.py --log-file \
  ../benchmark/e2e_perf.jsonl
```

References

[1] Hamdy Abdelkhalik, Yehia Arafa, Nandakishore Santhi, and Abdel-Hameed Badawy. 2022. Demystifying the Nvidia Ampere Architecture through Microbenchmarking and Instruction-level Analysis. arXiv:2208.11174 [cs.AR] https://arxiv.org/abs/2208.11174

[2] Meta AI. 2025. The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation. https://ai.meta.com/blog/llama-4-multimodal-intelligence/. Blog post, published April 5, 2025.

[3] AI@Meta. 2024. Llama 3 Model Card. https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md. (2024). https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md

[4] Anthropic. 2025. Claude Code: Deep coding at terminal velocity. https://www.anthropic.com/claude-code. Official documentation overview of Claude Code agentic coding tool.

[5] Payman Behnam, Yaosheng Fu, Ritchie Zhao, Po-An Tsai, Zhiding Yu, and Alexey Tumanov. 2025. RocketKV: Accelerating Long-Context LLM Inference via Two-Stage KV Cache Compression. In *Proceedings of the 42st International Conference on Machine Learning*. https://openreview.net/forum?id=RyOpooIxDF

[6] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher M De Sa. 2023. Quip: 2-bit quantization of large language models with guarantees. *Advances in Neural Information Processing Systems* 36 (2023), 4396–4429.

[7] Sumit Kumar Dam, Choong Seon Hong, Yu Qiao, and Chaoning Zhang. 2024. A complete survey on llm-based ai chatbots. *arXiv preprint arXiv:2406.16937* (2024).

[8] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691* (2023).

[9] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35 (2022), 16344–16359.

[10] Xin Luna Dong, Seungwhan Moon, Yifan Ethan Xu, Kshitiz Malik, and Zhou Yu. 2023. Towards next-generation intelligent assistants leveraging llm techniques. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5792–5793.

[11] Yitao Hu, Xiulong Liu, Guotao Yang, Linxuan Li, Kai Zeng, Zhixin Zhao, Sheng Chen, Laiping Zhao, Wenxin Li, and Keqiu Li. 2025. TightLLM: Maximizing Throughput for LLM Inference via Adaptive Offloading Policy. *IEEE Trans. Comput.* (2025).

[12] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-Coder Technical Report. arXiv:2409.12186 [cs.CL] https://arxiv.org/abs/2409.12186

[13] Jing Jin, Houfeng Wang, Hao Zhang, Xiaoguang Li, and Zhijiang Guo. 2024. DVD: Dynamic contrastive decoding for knowledge amplification in multi-document question answering. In *Proceedings of the 2024 conference on empirical methods in natural language processing*. 4624–4637.

[14] Jordan Juravsky, Bradley Brown, Ryan Ehrlich, Daniel Y Fu, Christopher Ré, and Azalia Mirhoseini. 2024. Hydragen: High-throughput llm inference with shared prefixes. *arXiv preprint arXiv:2402.05099* (2024).

[15] Aditya K Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. 2025. Pod-attention: Unlocking full prefill-decode overlap for faster llm inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 897–912.

[16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

[17] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. {InfiniGen}: Efficient generative inference of large language models with dynamic {KV} cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 155–172.

[18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[19] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 929–945.

[20] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of machine learning and systems* 6 (2024), 87–100.

[21] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434* (2024).

[22] Microsoft. 2025. AutoGen: A programming framework for agentic AI. https://github.com/microsoft/autogen. GitHub repository, supports multi-agent AI applications framework.

[23] NVIDIA. 2020. NVIDIA A100 Tensor Core GPU Architecture In-Depth. https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf. White paper, 82 pages.

[24] NVIDIA. 2023. Parallel Thread Execution ISA (PTX ISA), Version 9.0. https://docs.nvidia.com/cuda/parallel-thread-execution/. Online documentation (CUDA PTX ISA guide), last updated February 27, 2023.

[25] NVIDIA. 2025. CUDA C++ Best Practices Guide, Release 13.0. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/. Online documentation, includes downloadable PDF version (Aug 1, 2025).

[26] NVIDIA. 2025. CUDA C++ Programming Guide, Release 13.0. https://docs.nvidia.com/cuda/cuda-c-programming-guide/. Online documentation and PDF (NVIDIA Corporation, Aug 1, 2025, 596 pp.).

[27] NVIDIA. 2025. CUTLASS Quick Start Guide. https://docs.nvidia.com/cutlass/media/docs/cpp/quickstart.html. Accessed: August 20, 2025.

[28] NVIDIA. 2025. FasterTransformer: Transformers optimization framework for inference. https://github.com/NVIDIA/FasterTransformer.

[29] NVIDIA. 2025. Getting Started with CuTe. https://docs.nvidia.com/cutlass/media/docs/cpp/cute/00_quickstart.html. Online documentation, last updated August 7, 2025.

[30] NVIDIA. 2025. NVIDIA CUTLASS Documentation. https://docs.nvidia.com/cutlass/index.html. Online documentation hub, last updated August 7, 2025.

[31] NVIDIA. 2025. NVIDIA Nsight Compute: an interactive CUDA and OptiX profiler. https://developer.nvidia.com/nsight-compute. Online documentation, last updated August 2025; interactive kernel profiling tool with guided analysis for CUDA and OptiX.

[32] OpenAI. 2025. Prompt caching: Reduce latency and cost with prompt caching. https://platform.openai.com/docs/guides/prompt-caching.

[33] Zaifeng Pan, Yitong Ding, Yue Guan, Zheng Wang, Zhongkai Yu, Xulong Tang, Yida Wang, and Yufei Ding. 2025. FastTree: Optimizing Attention Kernel and Runtime for Tree-Structured LLM Inference. In *Eighth Conference on Machine Learning and Systems*.

[34] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading more storage for less computation—a {KVCache-centric} architecture for serving {LLM} chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. 155–170.

[35] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, et al. 2024. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *ACM Transactions on Storage* (2024).

[36] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems* 37 (2024), 68658–68685.

[37] Zhuoran Shen, Mingyuan Zhang, Haiyu Zhao, Shuai Yi, and Hongsheng Li. 2021. Efficient attention: Attention with linear complexities. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*. 3531–3539.

[38] Mingcong Song, Xinru Tang, Fengfan Hou, Jing Li, Wei Wei, Yipeng Ma, Runqiu Xiao, Hongjie Si, Dingcheng Jiang, Shouyi Yin, et al. 2024. Tackling the dynamicity in a production llm serving system with sota optimizations via hybrid prefill/decode/verify scheduling on efficient meta-kernels. *arXiv preprint arXiv:2412.18106* (2024).

[39] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*. 173–191.

[40] Gemma Team. 2025. Gemma 3. (2025). https://goo.gle/Gemma3Report

[41] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. 2025. KVCache Cache in the Wild: Characterizing and Optimizing KVCache Cache at a Large Cloud Provider. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association. https://www.usenix.org/conference/atc25/presentation/wang-jiahao

[42] Yaqing Wang, Quanming Yao, James T Kwok, and Lionel M Ni. 2020. Generalizing from a few examples: A survey on few-shot learning. *ACM computing surveys (csur)* 53, 3 (2020), 1–34.

[43] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.

[44] Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. 2025. Inference scaling laws: An empirical analysis of compute-optimal inference for LLM problem-solving. In *The Thirteenth International Conference on Learning Representations*.

[45] Jiale Xu, Rui Zhang, Yi Xiong, Cong Guo, Zihan Liu, Yangjie Zhou, Weiming Hu, Hao Wu, Changxu Shao, Ziqing Wang, et al. 2025. eLLM: Elastic Memory Management Framework for Efficient LLM Serving. *arXiv preprint arXiv:2506.15155* (2025).

[46] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[47] Jinwei Yao, Kaiqi Chen, Kexun Zhang, Jiaxuan You, Binhang Yuan, Zeke Wang, and Tao Lin. 2024. Deft: Decoding with flash tree-attention for efficient tree-structured llm inference. *arXiv preprint arXiv:2404.00242* (2024).

[48] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*. 94–109.

[49] Lu Ye, Ze Tao, Yong Huang, and Yang Li. 2024. ChunkAttention: Efficient Self-Attention with Prefix-Aware KV Cache and Two-Phase Partition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 11608–11620.

[50] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. 2025. Flashinfer: Efficient and customizable attention engine for llm inference serving. *arXiv preprint arXiv:2501.01005* (2025).

[51] Zihao Ye, Ruihang Lai, Bo-Ru Lu, Chien-Yu Lin, Size Zheng, Lequn Chen, Tianqi Chen, and Luis Ceze. 2024. Cascade Inference: Memory Bandwidth Efficient Shared Prefix Batch Decoding. https://flashinfer.ai/2024/02/02/cascade-inference.html

[52] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.

[53] Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Yongliang Shen, Ren Kan, Dongsheng Li, and Deqing Yang. 2024. Easytool: Enhancing llm-based agents with concise tool instruction. *arXiv preprint arXiv:2401.06201* (2024).

[54] Ted Zadouri, Hubert Strauss, and Tri Dao. 2025. Hardware-Efficient Attention for Fast Decoding. arXiv:2505.21487 [cs.LG] https://arxiv.org/abs/2505.21487

[55] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.

[56] Zhen Zheng, Xin Ji, Taosong Fang, Fanghao Zhou, Chuanjie Liu, and Gang Peng. 2024. Batchllm: Optimizing large batched llm inference with global prefix sharing and throughput-oriented token batching. *arXiv preprint arXiv:2412.03594* (2024).

[57] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.

[58] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. 2022. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems* 35 (2022), 7103–7114.

[59] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, et al. 2025. {NanoFlow}: Towards Optimal Large Language Model Serving Throughput. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 749–765.

[60] Lei Zhu, Xinjiang Wang, Wayne Zhang, and Rynson Lau. 2024. RelayAttention for Efficient Large Language Model Serving with Long System Prompts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 4945–4957.

[61] Zayd Muhammad Kawakibi Zuhri, Muhammad Farid Adilazuarda, Ayu Purwarianti, and Alham Fikri Aji. 2024. Mlkv: Multi-layer key-value heads for memory efficient transformer decoding. *arXiv preprint arXiv:2406.09297* (2024).